

Paimon Metastore

模块架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：Metastore (paimon-core/metastore)

分支：release-1.3.1

生成工具：AI Architecture Analyzer

生成日期：2026年03月29日

第1章 模块概述

1.1 模块定位

Metastore 模块位于 paimon-core 中的 org.apache.paimon.metastore 包下，是 Apache Paimon 提交流程中与外部元数据存储（如 Hive Metastore）进行交互的回调层。该模块的核心职责是在表数据提交（Commit）和标签（Tag）操作前后，将分区变更信息同步到外部元数据服务中，确保 Paimon 表的分区信息与外部 Metastore 保持一致。

此外，该模块还包含 Chain Table（链式表）场景下的提交前校验回调和覆盖写提交回调，以及基于 Compaction 等待的可见性保障回调。模块通过实现 CommitCallback、CommitPreCallback、TagCallback 三大回调接口，嵌入到 Paimon 的提交生命周期中。

1.2 核心目标

- 分区同步：在数据提交后，将新增分区自动注册到外部 Metastore（如 Hive），实现分区的自动发现
- Tag 分区映射：在 Tag 创建/删除时，将 Tag 名称映射为分区并同步至 Metastore
- Chain Table 安全保障：在链式表的分区删除和覆盖写场景下进行前置校验与后置同步
- 可见性等待：在批处理场景下等待 Compaction 完成，确保数据对下游查询可见

1.3 源码文件一览

文件名	行数	职责描述
AddPartitionCommitCallback.java	105	提交后回调：将新增分区注册到 Metastore，带 Guava Cache 去重
AddPartitionTagCallback.java	66	Tag 回调：Tag 创建/删除时同步分区到 Metastore
ChainTableCommitPreCallback.java	208	提交前回调：Chain Table 分区删除的安全性校验
ChainTableOverwriteCommitCallback.java	121	提交后回调：Chain Table 覆盖写时同步截断快照分支
TagPreviewCommitCallback.java	59	提交后回调：Tag 预览场景下基于时间提取 Tag 并创建分区
VisibilityWaitCallback.java	149	提交后回调：等待 Compaction 完成以保证数据可见性

第2章 核心架构设计

2.1 整体架构图

Metastore 模块采用回调接口分层架构，共分为三层：接口层定义了 CommitCallback、CommitPreCallback、TagCallback 三大回调契约；实现层包含 6 个具体回调类；依赖层对接 PartitionModification、ChainTableUtils、TagPreview 等外部服务。

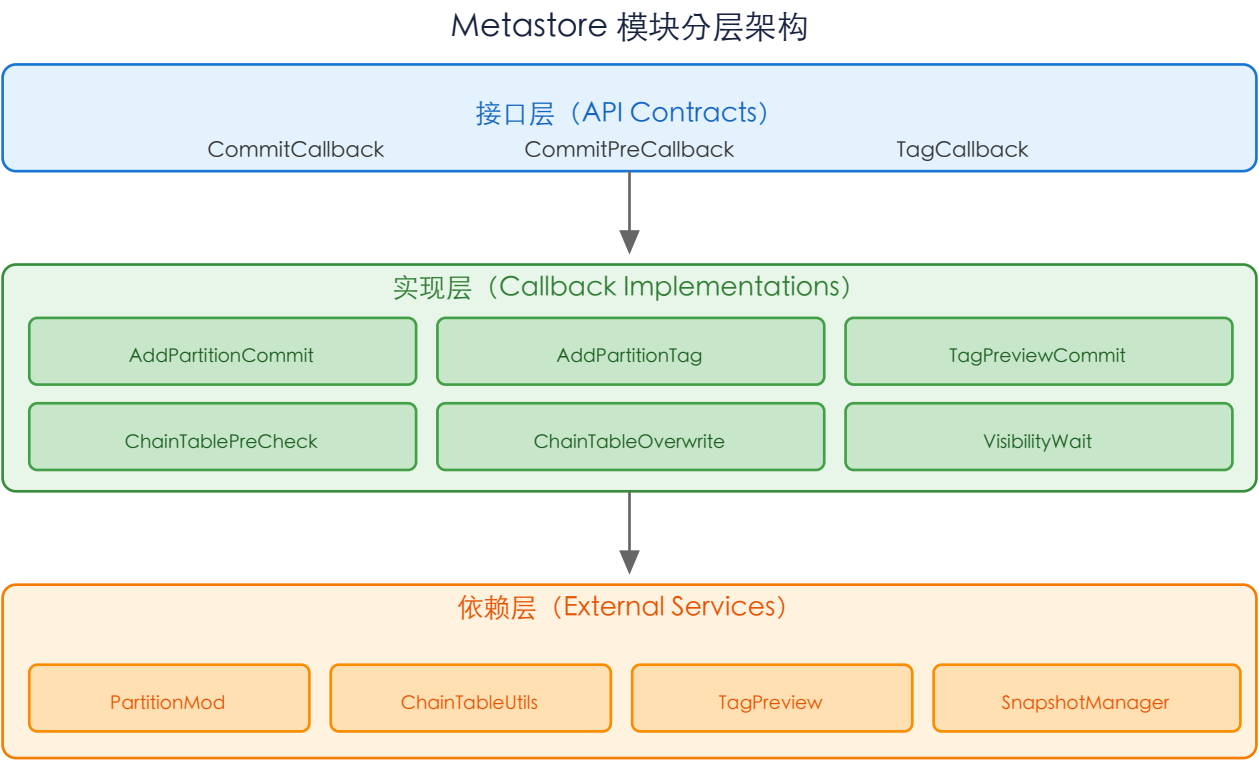


图2-1 Metastore 模块分层架构图

2.2 类继承关系

所有回调类均实现了 `AutoCloseable` 接口（通过各自的父接口继承），支持资源释放。类继承关系如下：

类名	实现接口	继承关系说明
AddPartitionCommitCallback	CommitCallback	实现 call/retry/close 方法，提交后注册分区
AddPartitionTagCallback	TagCallback	实现 notifyCreation/notifyDeletion/close 方法
ChainTableCommitPreCallback	CommitPreCallback	实现 call/close 方法，提交前校验
ChainTableOverwriteCommitCallback	CommitCallback	实现 call/retry/close 方法，覆盖写同步

类名	实现接口	继承关系说明
TagPreviewCommitCallback	CommitCallback	实现 call/retry/close 方法，组合 AddPartitionTagCallback
VisibilityWaitCallback	CommitCallback	实现 call/retry/close 方法，等待 Compaction

2.3 接口契约说明

2.3.1 CommitCallback 接口

CommitCallback 是提交后回调接口，继承 AutoCloseable。在 ManifestCommittable 列表提交成功后被调用。接口要求实现必须是幂等的，因为失败重试场景下可能被多次调用。

- call(Context) — 正常提交成功后的回调，Context 包含 baseFiles、deltaFiles、indexFiles、snapshot 和 identifier
- retry(ManifestCommittable) — 重试场景下的回调
- setTable(FileStoreTable) — 可选的 default 方法，用于设置表引用

2.3.2 CommitPreCallback 接口

CommitPreCallback 是提交前回调接口。在 Snapshot 提交之前被调用，用于额外的校验或记账操作。如果 call 方法抛出 RuntimeException，提交将被中止。实现应快速执行且幂等。

- call(baseFiles, deltaFiles, indexFiles, snapshot) — 提交前校验钩子

2.3.3 TagCallback 接口

TagCallback 是 Tag 操作回调接口。在 Tag 创建/删除后被调用。不保证一定会被调用。

- notifyCreation(tagName) — Tag 创建通知
- notifyCreation(tagName, snapshotId) — 带 snapshotId 的创建通知 (Iceberg 兼容)
- notifyDeletion(tagName) — Tag 删除通知

第3章 核心类设计模式详解

3.1 AddPartitionCommitCallback — 策略模式 + 缓存模式

策略模式 (Strategy) : AddPartitionCommitCallback 实现了 CommitCallback 接口，作为提交后分区同步的具体策略。调用方无需关心分区注册的具体实现，只需通过接口调用回调。

缓存模式 (Cache-Aside) : 该类内部使用 `Guava` `Cache` (`CacheBuilder`) 维护已注册分区的缓存，设置了 30 分钟过期和 300 最大容量，使用 `softValues` 避免极端情况下的内存溢出。在 `addPartitions` 方法中先查缓存过滤已知分区，只对新分区调用 `PartitionModification.createPartitions`，并在成功后更新缓存。

关键方法：

- `call(Context)` — 从 `deltaFiles` 中过滤 `FileKind.ADD` 的条目，收集分区集合后调用 `addPartitions`
- `retry(ManifestCommittable)` — 从 `committable` 的 `fileCommittables` 中提取分区并重新注册
- `addPartitions(Set)` — 核心逻辑：缓存查询 -> 过滤新分区 -> 批量创建 -> 更新缓存

3.2 AddPartitionTagCallback — 策略模式 + 命令模式

策略模式 (Strategy) : 实现 TagCallback 接口，作为 Tag 事件的分区同步策略。

命令模式 (Command) : `notifyCreation` 和 `notifyDeletion` 方法将 Tag 名称封装为分区规格 (`LinkedHashMap`)，然后分别委托给 `PartitionModification` 的 `createPartitions` 和 `dropPartitions` 执行。每个通知方法相当于一个具体命令。

- `notifyCreation(tagName)` — 将 `tagName` 映射为 `partitionField:tagName` 的分区规格并创建
- `notifyDeletion(tagName)` — 将 `tagName` 映射为同样的分区规格并删除

3.3 TagPreviewCommitCallback — 组合模式 + 委托模式

组合模式 (Composite) : TagPreviewCommitCallback 内部组合了 AddPartitionTagCallback 和 TagPreview 两个对象，将提交后回调的能力与 Tag 预览的时间提取能力组合在一起。

委托模式 (Delegation) : `call` 和 `retry` 方法先通过 `TagPreview.extractTag` 提取 Tag 名称，再将创建操作委托给 `AddPartitionTagCallback.notifyCreation`，`close` 方法也直接委托给 `tagCallback.close`。

3.4 ChainTableCommitPreCallback — 模板方法模式 + 守卫检查

模板方法模式 (Template Method) : `call` 方法定义了一个固定的执行模板：首先进行三重守卫检查（是否为 Chain Table 快照分支、是否为

OVERWRITE 提交、是否为纯删除提交），然后对每个变更分区执行查找前驱/后继分区 + 查询 Delta 分区 + 判断是否可删除的校验流程。

守卫检查模式（Guard Clause）：三个 if-return 短路条件使得核心校验逻辑仅在满足所有前置条件时执行，提升代码可读性。

- findPreSnapshotPartition — 在有序分区列表中查找目标分区的前驱
- findNextSnapshotPartition — 在有序分区列表中查找目标分区的后继
- isPureDeleteCommit — 判断所有 delta 和 index 文件是否均为 DELETE 类型

3.5 ChainTableOverwriteCommitCallback — 策略模式 + 幂等设计

策略模式：实现 CommitCallback 接口，专门处理 Chain Table Delta 分支上的 OVERWRITE 提交事件。

幂等设计：retry 方法为 No-op，因为在快照分支上截断相同分区是安全的幂等操作。call 方法中收集 OVERWRITE 涉及的分区的值，转换为分区值映射后通过 BatchTableCommit.truncatePartitions 在快照分支上执行截断。

3.6 VisibilityWaitCallback — 轮询等待模式（Polling）

轮询等待模式：waitForCompaction 方法采用经典的轮询-休眠-超时模式。在循环中不断检查最新 Snapshot 中是否仍存在待压缩文件，直到所有追踪的文件都已被 Compaction 处理或超时。

条件过滤模式：shouldBeTracked 方法根据 deletionVectorsEnabled 和 bucket 模式决定哪些文件需要追踪。只有 FileKind.ADD 且满足条件（DV 启用时 level=0，或 POSTPONE_BUCKET 模式）的文件才会被等待。

- call(Context) — 仅在 BatchWriteBuilder.COMMIT_IDENTIFIER

场景下生效，收集待追踪文件后调用等待

- waitForCompaction — 轮询逻辑：检查最新快照 -> 比对文件名 -> 休眠 -> 超时抛异常
- stillInLatest — 读取快照文件列表，检查追踪文件是否仍存在

第4章 核心流程图

4.1 分区注册流程 (AddPartitionCommitCallback)

当数据提交成功后，AddPartitionCommitCallback

被触发，从提交的文件变更中提取新增分区，经过缓存过滤后批量注册到外部 Metastore。

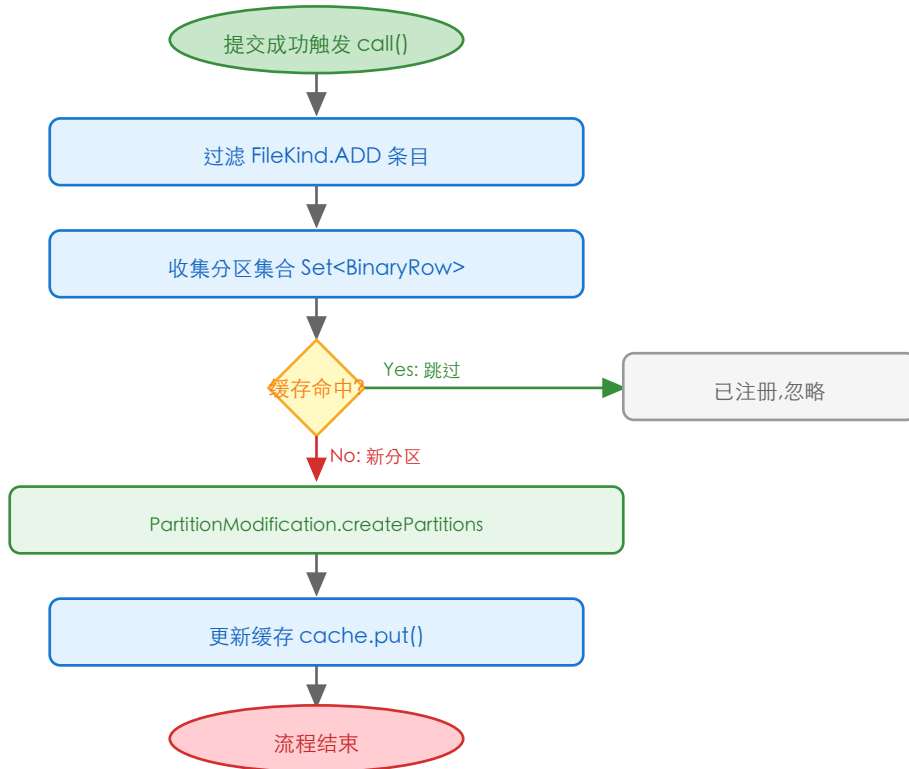


图4-1 分区注册流程图

4.2 Chain Table

分区删除校验流程（ChainTableCommitPreCallback）

在 Chain Table 的快照分支上执行 OVERWRITE 提交（删除分区）前，ChainTableCommitPreCallback 执行安全性校验，确保不会破坏 Chain Table 的数据完整性。

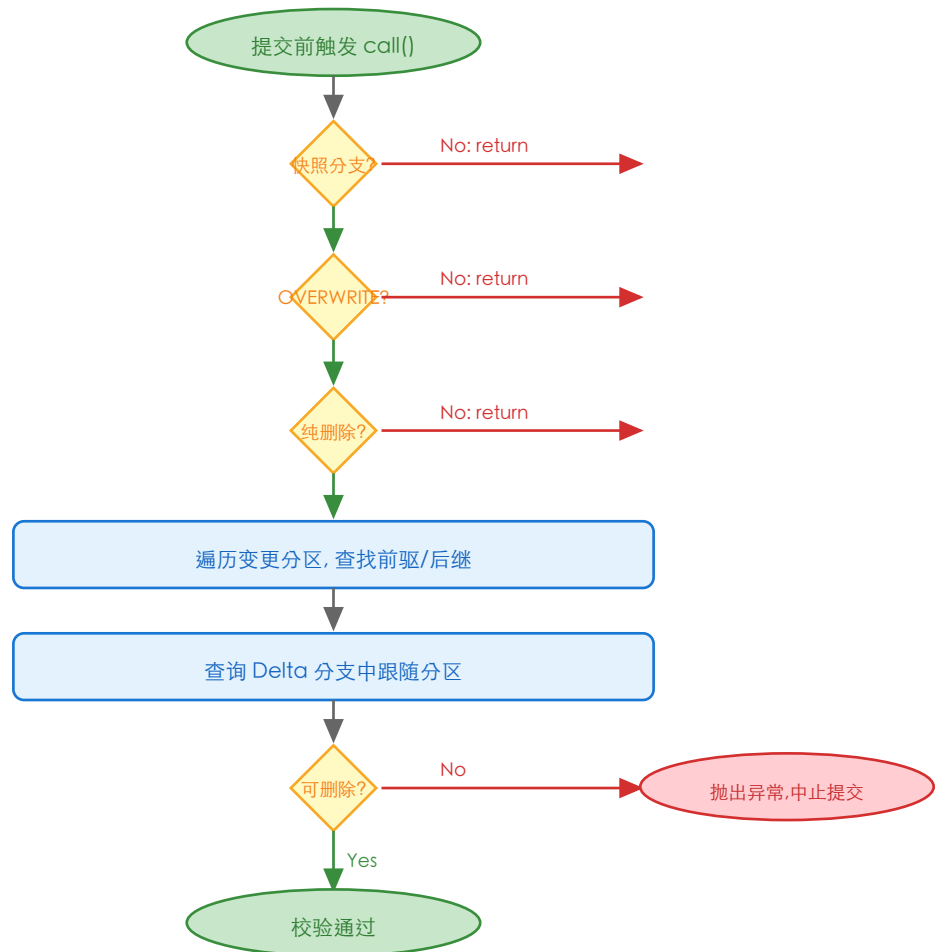


图4-2 Chain Table 分区删除校验流程图

4.3 可见性等待流程 (VisibilityWaitCallback)

在批处理场景下，VisibilityWaitCallback 等待新增的 Level-0 文件或 POSTPONE_BUCKET 文件被 Compaction 处理后消失，确保数据对下游查询可见。

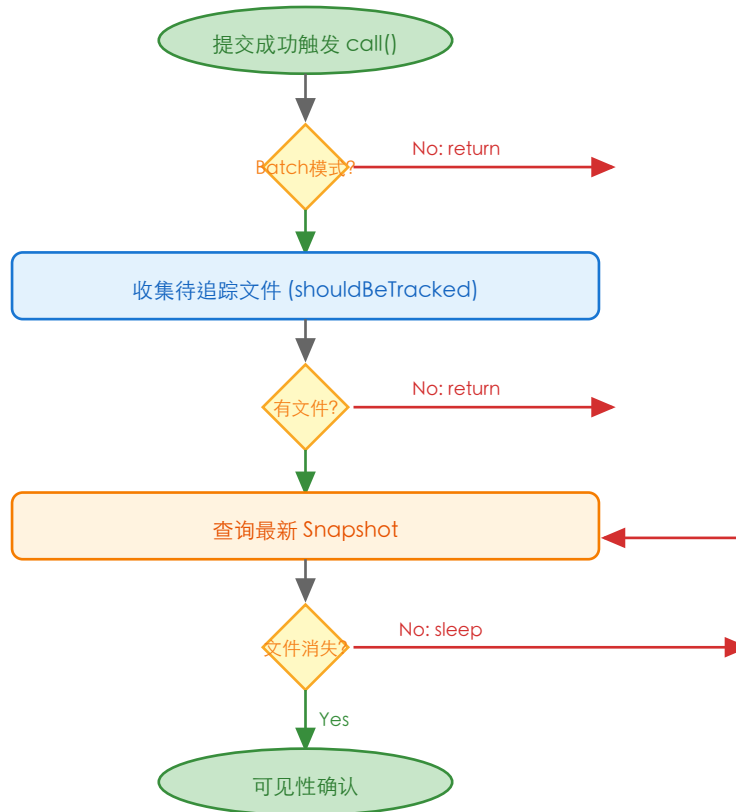


图4-3 可见性等待流程图

第5章 核心时序图

5.1 提交后分区注册时序图

展示数据提交成功后，AddPartitionCommitCallback 如何与 Guava Cache 和 PartitionModification 交互完成分区注册。

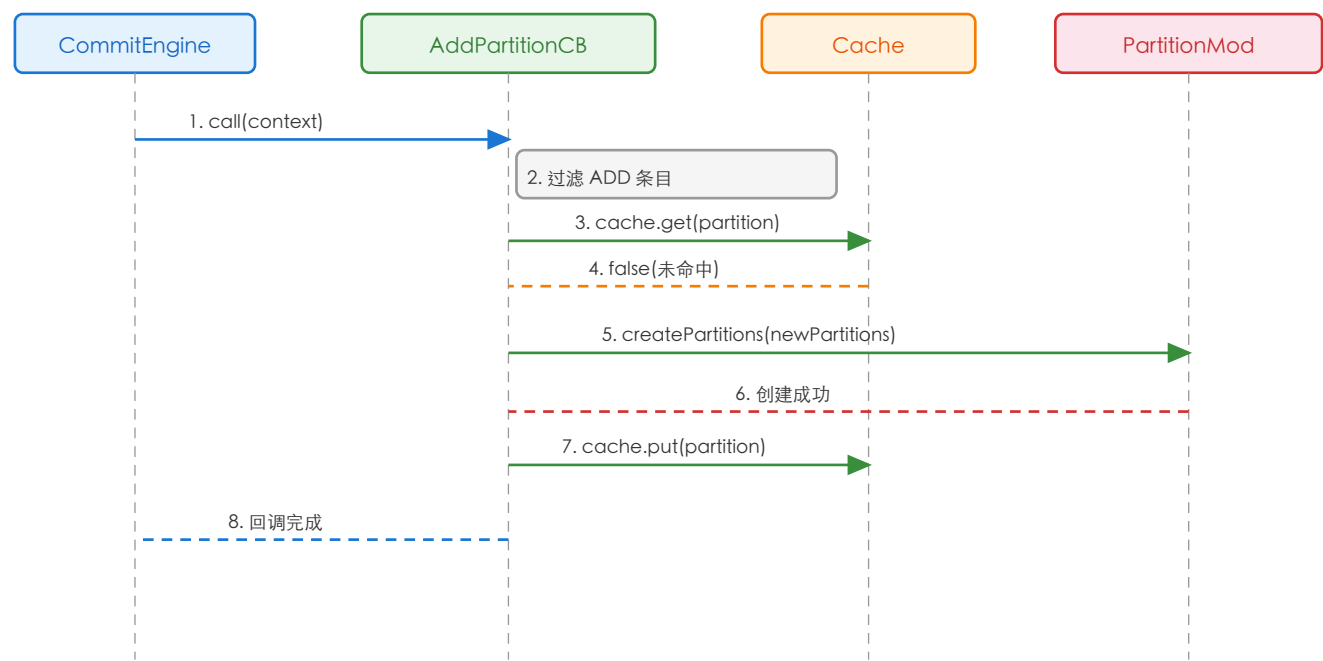


图5-1 提交后分区注册时序图

5.2 Chain Table 覆盖写同步时序图

展示 ChainTableOverwriteCommitCallback 在 Delta 分支覆盖写提交后，如何在快照分支上截断对应分区。

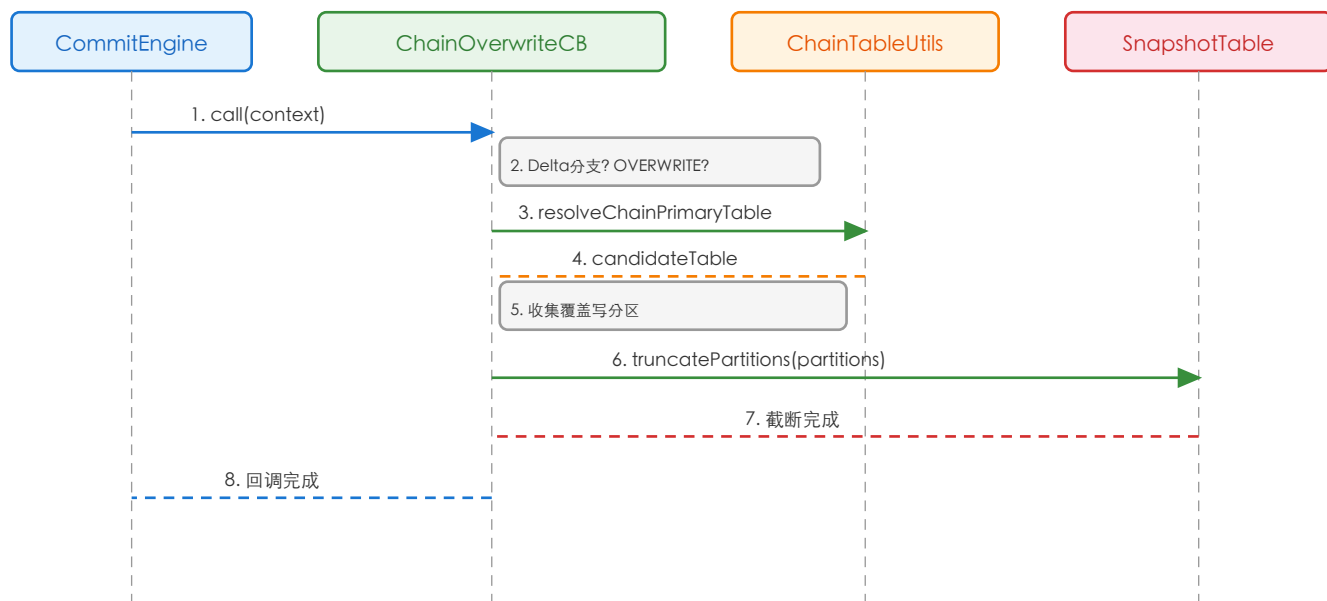


图5-2 Chain Table 覆盖写同步时序图

5.3 可见性等待时序图

展示 VisibilityWaitCallback 在批处理提交后如何轮询等待 Compaction 完成。

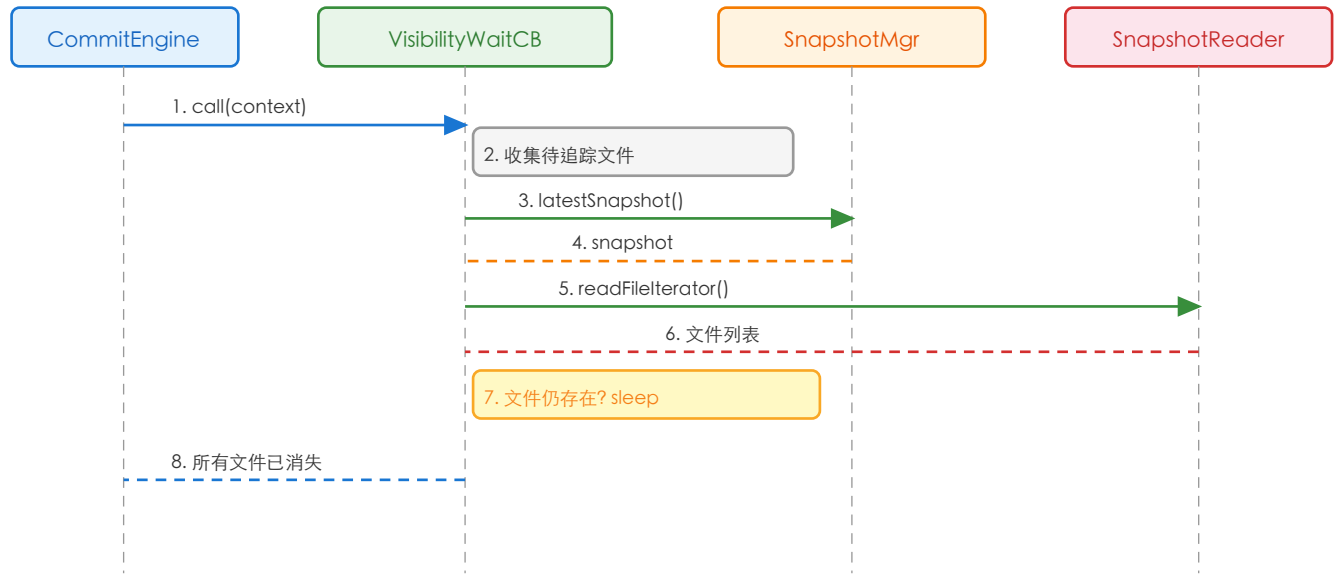


图5-3 可见性等待时序图

第6章 关键场景调用链

6.1 场景一：常规数据提交后分区注册

触发条件：数据写入提交成功后，CommitEngine 触发注册的 CommitCallback 列表

调用链路：

步骤	类.方法	说明
1	CommitEngine.commit()	提交成功后遍历 callbacks
2	AddPartitionCommitCallback.call(context)	接收 Context，提取 deltaFiles
3	context.deltaFiles.stream().filter(ADD)	过滤 FileKind.ADD 的 ManifestEntry
4	ManifestEntry.partition()	提取每个文件的分区信息 BinaryRow
5	cache.get(partition, () -> false)	Guava Cache 查询是否已注册
6	PartitionModification.createPartitions()	调用 Catalog 创建新分区
7	cache.put(partition, true)	更新缓存标记已注册

6.2 场景二：Tag 创建后分区同步

触发条件：用户创建 Tag 或自动 Tag 机制触发

调用链路：

步骤	类.方法	说明
1	TagManager.createTag()	Tag 创建完成后通知回调
2	AddPartitionTagCallback.notifyCreation(tagName)	接收 Tag 名称
3	LinkedHashMap.put(partitionField, tagName)	构建分区规格映射
4	PartitionModification.createPartitions()	通过 Catalog 在 Metastore 创建分区

6.3 场景三：Tag 预览提交回调

触发条件：流式提交场景下，TagPreviewCommitCallback 从 watermark 中提取 Tag 信息

调用链路：

步骤	类.方法	说明
1	CommitEngine.commit()	提交成功后遍历 callbacks
2	TagPreviewCommitCallback.call(context)	获取 snapshot watermark
3	TagPreview.extractTag(currentMillis, watermark)	根据时间和水位提取 Tag 名称
4	TagTimeExtractor.extract(timeMilli, watermark)	从时间/水位中提取 LocalDateTime
5	TagPeriodHandler.normalizeToPreviousTag()	将时间规范化到上一个周期边界
6	TagPeriodHandler.nextTagTime()	计算下一个 Tag 时间点
7	AddPartitionTagCallback.notifyCreation(tag)	委托创建对应分区

6.4 场景四：Chain Table OVERWRITE 分区删除校验

触发条件：在 Chain Table 快照分支上执行 OVERWRITE 删除分区时的前置校验

调用链路：

步骤	类.方法	说明
1	FileStoreCommit.commit()	提交前触发 PreCallback
2	ChainTableCommitPreCallback.call(...)	接收文件列表和快照
3	ChainTableUtils.isScanFallbackSnapshotBranch()	检查是否为快照分支
4	isPureDeleteCommit(deltaFiles, indexFiles)	检查是否为纯删除提交
5	ManifestEntryChanges.changedPartitions()	提取变更分区列表
6	ChainTableUtils.resolveChainPrimaryTable()	解析 Chain 主表
7	table.newSnapshotReader().partitionEntries()	读取快照分支所有分区并排序
8	findPreSnapshotPartition()	查找目标分区的前驱快照分区
9	findNextSnapshotPartition()	查找目标分区的后继快照分区
10	ChainTableUtils.createTriangularPredicate()	构建三角谓词查询 Delta 跟随分区
11	deltaSnapshotReader.partitionEntries()	查询 Delta 分支中的跟随分区
12	canDrop 判断	若无 Delta 跟随分区或存在前驱分区则可删

6.5 场景五：Chain Table 覆盖写后截断快照分支

触发条件：在 Chain Table Delta 分支上完成 OVERWRITE 提交后

调用链路：

步骤	类.方法	说明
1	CommitEngine.commit()	提交成功后遍历 callbacks
2	ChainTableOverwriteCommitCallback.call(context)	接收 Context
3	ChainTableUtils.isScanFallbackDeltaBranch()	检查当前为 Delta 分支
4	context.deltaFiles.stream().map(partition).distinct()	收集覆盖写涉及的分区
5	partitionComputer.generatePartValues()	将 BinaryRow 转换为 Map 分区值

步骤	类.方法	说明
6	snapshotTable.newBatchWriteBuilder().newCommit()	获取快照分支提交器
7	BatchTableCommit.truncatePartitions()	在快照分支上截断对应分区

6.6 场景六：批处理可见性等待

触发条件：批处理（identifier == COMMIT_IDENTIFIER）提交成功后

调用链路：

步骤	类.方法	说明
1	CommitEngine.commit()	提交成功后遍历 callbacks
2	VisibilityWaitCallback.call(context)	检查 identifier 是否为批处理
3	shouldBeTracked(entry)	判断文件是否需追踪(DV level-0 或 POSTPONE_BUCKET)
4	waitForCompaction(snapshot, names, partitions)	进入轮询等待循环
5	table.snapshotManager().latestSnapshot()	获取最新快照
6	stillInLatest(snapshot, namesToTrack, ...)	在最新快照中检查文件是否仍存在
7	snapshotReader.readFileIterator()	遍历快照分区文件
8	Thread.sleep(checkInterval)	休眠后继续轮询直到超时或文件消失

第7章 配置与扩展

7.1 相关配置属性

配置项 / 常量	来源	说明
Guava Cache expireAfterAccess	AddPartitionCommitCallback	30 分钟过期，避免缓存膨胀
Guava Cache maximumSize	AddPartitionCommitCallback	最大 300 条缓存，使用 softValues
visibility.callback.timeout	CoreOptions	可见性等待超时时间
visibility.callback.check-interval	CoreOptions	可见性检查间隔
deletion-vectors.enabled	CoreOptions	是否启用删除向量，影响文件追踪逻辑
scan.fallback-branch (delta/snapshot)	CoreOptions	Chain Table 的 Delta/Snapshot 分支名称
tag-to-partition-preview	CoreOptions	Tag 预览模式配置（NONE/BATCH/其他）

7.2 扩展点

Metastore 模块的扩展主要通过以下方式实现：

- 实现 CommitCallback 接口：可添加新的提交后回调，如发送通知、更新统计等
- 实现 CommitPreCallback 接口：可添加新的提交前校验，如权限检查、配额验证等
- 实现 TagCallback 接口：可添加新的 Tag 事件处理器，如审计日志、Iceberg Tag 同步等
- 实现 PartitionModification 接口：可对接不同的 Metastore 实现（已有 Catalog 委托的工厂方法）

第8章 设计亮点与总结

8.1 设计亮点

- 回调接口抽象精准：CommitCallback（提交后）、CommitPreCallback（提交前）、TagCallback（Tag 事件）三个接口清晰划分了回调时机，职责边界明确。
- 幂等性设计：所有 CommitCallback 实现均考虑了幂等性，retry 方法安全可重复调用。AddPartitionCommitCallback 通过 Guava Cache 避免重复注册，ChainTableOverwriteCommitCallback 的截断操作天然幂等。
- 守卫检查模式：ChainTableCommitPreCallback 使用三重 if-return 守卫检查，使核心逻辑仅在满足所有前置条件时执行，代码清晰可读。
- 组合委托设计：TagPreviewCommitCallback 将 CommitCallback 与 TagCallback 能力组合，通过委托 AddPartitionTagCallback 实现 Tag 预览分区创建，代码复用度高。
- Chain Table 数据完整性保护：CommitPreCallback 在分区删除前执行严格的完整性校验，验证前驱/后继分区关系和 Delta 跟随分区状态，防止数据链断裂。

8.2 设计限制

- VisibilityWaitCallback 轮询开销：基于 Thread.sleep 的轮询等待在高并发场景下可能造成线程资源浪费，可考虑事件驱动或异步回调优化。
- 异常处理粗粒度：部分回调类（如 AddPartitionCommitCallback.addPartitions）使用 catch(Exception) + RuntimeException 包装，异常类型信息丢失。
- 缓存一致性风险：AddPartitionCommitCallback 的 Guava Cache 为本地内存缓存，在多线程实例场景下可能导致重复注册（但由于 Metastore createPartition 通常是幂等的，影响可控）。

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	分层清晰，接口抽象合理，每个类职责单一
设计模式运用	★★★★☆	灵活运用策略、组合、委托、轮询等模式
可扩展性	★★★★★	基于接口编程，新增回调只需实现接口即可
幂等与安全性	★★★★★	全面考虑重试幂等性和 Chain Table 数据完整性
性能考量	★★★★☆	缓存优化效果好，轮询等待有优化空间
文档完整度	★★★☆☆	接口 Javadoc 完善，实现类注释可再丰富